JANMITRA

Civic Issue Reporting & Management System

Nagpur Municipal Corporation | 4 Categories | 4 Wards | 4 Departments

Team ResolveIT

Development Document

Tech stack, setup, code structure, deployment guide

CITIZEN PORTAL	Report Issue | Track Status | Verify Resolution
BACKEND (Django)	REST API | Auth | Business Logic | AI Module
AI VALIDATION	Fake Detection (EXIF+ELA+HuggingFace) | YOLO Relevance Check
DATABASE	PostgreSQL + PostGIS | 4 Categories | 4 Wards | 4 Depts
DEPT. DASHBOARD	PWD | Solid Waste | Water Works | Electrical

Anas Shaikh | Anurag Deshmukh | Om Gadekar | Faizan Rayeen

September 2026 | B.Tech Final Year Project

1. Revised Technology Stack

INFO This tech stack was carefully reviewed and simplified. Every tool has a clear purpose. Nothing was kept just because it is popular. The goal: practical, consistent, easy to build, suitable for a college project, and zero cost.

1.1 Frontend

Technology	Version	Role	Why Chosen
React.js	18.x	UI framework	Component-based, works for all 4 portals in one codebase
Tailwind CSS	3.x	Styling	Utility-first, fast development, responsive built-in
React Router DOM	v6	Routing	Client-side navigation + ProtectedRoute for role-based access
React Context API	18.x	Auth state	Built into React. Simpler than Zustand for our scope.
Axios	Latest	HTTP client	JWT interceptor for auto token refresh. Minimal setup.
Leaflet	1.9	Maps	Free, no API key. OpenStreetMap tiles. Works offline.
React-Leaflet	4.x	React map components	Wraps Leaflet for React. Clean API.
Recharts	2.x	Charts	React-native chart library. Simple API for our dashboard.
Firebase JS SDK	10.x	Push notifications	Receive FCM push in browser. Service worker for background.

UPDATED Zustand removed: React Context API is sufficient for auth state (token + user + role). Adding Zustand would be unnecessary complexity for a 4-person college project.

1.2 Backend

Technology	Version	Role	Why Chosen
Python	3.11	Language	Same language as AI libs (YOLO, OpenCV, HuggingFace). No extra service needed.
Django	4.2 LTS	Framework	LTS version. ORM, migrations, admin panel, URL routing all included.
Django REST Framework	3.15	REST APIs	Serializers, viewsets, permissions. Standard Django REST library.
DRF SimpleJWT	5.3	Auth	JWT access + refresh tokens. Easy RBAC with permission classes.
django-cors-headers	4.4	CORS	Allows React (port 5173) to call Django (port 8000) in development.
django-environ	0.11	Config	Load DATABASE_URL, REDIS_URL, etc. from .env file securely.
Gunicorn	22.x	Production	Replaces Django runserver in production on Render.
psycopg2-binary	2.9	DB driver	Python-PostgreSQL connection driver.

1.3 Database

Technology	Version	Role	Why Chosen
PostgreSQL	15	Primary DB	Supports PostGIS. Best Django ORM support. Free on Render.
PostGIS	3.3	Spatial extension	GPS coordinate storage + 100m duplicate detection in one SQL query. Ward polygon if ne

UPDATED

Ward detection simplified: Citizen selects their ward from a dropdown (Ward 1-4). No automatic GPS-to-ward polygon lookup needed. This removes complexity while being equally accurate for 4 fixed wards.

1.4 AI / ML

Technology	Version	Role	Why Chosen
Pillow	10.4	EXIF reading	Read camera metadata from images. Layer 1 fake detection step 1.
OpenCV (headless)	4.10	ELA analysis	Compare image compression patterns. Layer 1 step 2. Headless = no display needed c
NumPy	1.26	Array math	Pixel difference calculation for ELA. Required by OpenCV.
Transformers	4.44	AI fake detector	HuggingFace pre-trained model: real vs AI-generated classifier. Layer 1 step 3.
PyTorch	2.3	ML runtime	Required by Transformers. Not used directly.
Ultralytics	8.2	YOLOv8 object detect	Layer 2: checks if image is a real civic scene. Fine-tuned on civic dataset.

1.5 Background Processing

Technology	Version	Role	Why Chosen
Celery	5.4	Task queue	Runs AI pipeline async. Citizen submits and moves on. Celery processes in background.
Redis	7.x	Message broker	Celery needs a broker. Redis is simplest and fastest. Free on Redis Cloud (30MB).
Firebase Admin	6.5	Push from Django	Send FCM push notifications from backend to browser. Free Firebase Spark plan.

1.6 Deployment & DevOps

Technology	Role	Why Chosen
Docker + Docker Compose	Local development	Celery+PostGIS work on Linux. Docker gives Linux on all Windows dev machines. One comm
Render Free Tier	Cloud hosting	Free web service, background worker, static site, PostgreSQL. Zero cost for prototype.
Redis Cloud	Hosted Redis	Render has no free Redis. Redis Cloud gives 30MB free. Enough for our task queue.
UptimeRobot	Render sleep fix	Pings backend every 10 min. Keeps Render free service awake during demo.
Git + GitHub	Version control	Standard. Free private repos. Each member works on feature branches.

1.7 Tools NOT Used (Removed)

Removed Tool	Reason for Removal
Zustand	React Context API is sufficient for our auth state. Zustand adds a dependency without extra benefit at this scale.
geojson.io wards	Ward system is simplified to a dropdown selection. No ward polygon GeoJSON needed for 4 wards.
Kaggle API	Roboflow has all needed civic datasets. Kaggle API adds setup complexity.
Stable Diffusion	For fake image generation in training: simpler to use existing fake image datasets on HuggingFace.
Multiple DB tools	Only PostgreSQL + PostGIS. No Redis as a cache, no SQLite fallback. One DB system.

2. Project Folder Structure

Backend

```
backend/
  resolveIT/
    settings/base.py         # Shared settings (INSTALLED_APPS, DRF, JWT, Celery)
    settings/local.py        # Dev: DEBUG=True, SQLite or local PG
    settings/production.py   # Render: DEBUG=False, env-based DB URL
    celery.py                # Celery app + beat schedule
    urls.py                  # Root URL routing
  accounts/
    models.py                # Custom User: mobile, role, ward_id, dept_id, designation
    serializers.py           # Registration, Login, WorkerRegistration serializers
    permissions.py           # IsCitizen, IsWorker, IsDeptOfficer, IsAdmin
    views.py                 # Register, Login, Refresh, WorkerCreate (admin only)
    urls.py
  complaints/
    models.py                # Category(4), Ward(4), Department(4), Complaint, Image, History
    validators.py            # validate_worker_assignment(worker, complaint) function
    tasks.py                 # process_complaint_ai(), route_complaint(), auto_close()
    views.py                 # ComplaintViewSet, WorkerTaskView, AssignWorkerView, VerifyView
    ai_module/
      exif_checker.py        # Layer 1: EXIF metadata check
      ela_checker.py         # Layer 1: Error Level Analysis
      fake_detector.py       # Layer 1: HuggingFace AI fake detector
      yolo_validator.py      # Layer 2: YOLOv8 relevance check
      duplicate_detector.py  # PostGIS 100m same-category-ward duplicate
      priority_engine.py     # Weighted 4-factor score -> Low/Med/High/Critical
      models/janmitra.pt     # Fine-tuned YOLO model file
    urls.py
  dashboard/
    views.py                 # KPI, WardStats, Trends, Workers, HighPriority, Validation
    serializers.py           # Dashboard response serializers
    urls.py
  notifications/
    models.py                # Notification model
    fcm_service.py           # Firebase Admin: send_push(user_id, title, body)
    tasks.py                 # send_push_notification Celery task
  requirements.txt
  Dockerfile
  .env.example
  manage.py
```

Frontend

```
frontend/
  src/
    context/AuthContext.jsx   # React Context: token, user, role, login, logout
    api/client.js             # Axios instance + JWT interceptor + auto-refresh
    components/
      StatusBadge.jsx         # Coloured pill for complaint status
      PriorityBadge.jsx       # Coloured pill for priority level
      CategoryBadge.jsx       # Icon + colour for each of 4 categories
      WardBadge.jsx           # Ward 1-4 badge
      ImageUploader.jsx       # Multi-photo upload with EXIF spinner + error
      LeafletMap.jsx          # Reusable Leaflet map (drag pin + display)
      KPICard.jsx             # Dashboard stat card
      DeptHeader.jsx          # Dept. dashboard header banner
    pages/
      auth/LoginPage.jsx      # Login + Register (all roles)
      citizen/
        Home.jsx              # Citizen home
        SubmitForm.jsx        # Complaint submission form
        MyComplaints.jsx      # List with filters
        ComplaintDetail.jsx   # Status timeline + verify
      worker/
        TaskList.jsx          # Priority-sorted task list (dept+ward filtered)
        TaskDetail.jsx        # Task detail + proof upload
      officer/
        Dashboard.jsx         # Full dept. dashboard
        AssignWorker.jsx      # Assign worker with dept+ward validation
        ValidationQueue.jsx   # AI low-confidence queue
      admin/
        CrossDeptDashboard.jsx# All 4 depts comparison
        WorkerManagement.jsx  # List, register, edit workers
    App.jsx                   # Router + ProtectedRoute HOC
    main.jsx
  public/firebase-messaging-sw.js
  tailwind.config.js
  vite.config.js
  package.json
  Dockerfile
```

3. Key Code Samples

3.1 Custom User Model (accounts/models.py)

```
class User(AbstractBaseUser, PermissionsMixin):
    id           = models.UUIDField(primary_key=True, default=uuid.uuid4)
    full_name    = models.CharField(max_length=100)
    mobile       = models.CharField(max_length=15, unique=True)
    email        = models.EmailField(blank=True)
    role         = models.CharField(max_length=20, choices=ROLES, default='citizen')
    ward         = models.ForeignKey('complaints.Ward', null=True, on_delete=models.SET_NULL)
    department   = models.ForeignKey('complaints.Department', null=True, on_delete=models.SET_NULL)
    designation  = models.CharField(max_length=100, blank=True)
    employee_id  = models.CharField(max_length=50, blank=True)
    status       = models.CharField(max_length=20, default='active')
    fcm_token    = models.TextField(blank=True)
    is_active    = models.BooleanField(default=True)
    registered_at = models.DateTimeField(auto_now_add=True)
    USERNAME_FIELD = 'mobile'
```

3.2 Worker Assignment Validator (complaints/validators.py)

```
from rest_framework.exceptions import ValidationError

def validate_worker_assignment(worker, complaint, override=False):
    '''Enforce dept + ward match rule for worker assignment.'''
    errors = []
    if worker.department_id != complaint.department_id:
        errors.append(
            f'Worker dept ({worker.department}) does not match '
            f'complaint dept ({complaint.department})'
        )
    if worker.ward_id != complaint.ward_id:
        errors.append(
            f'Worker ward ({worker.ward}) does not match '
            f'complaint ward ({complaint.ward})'
        )
    if errors and not override:
        raise ValidationError({'assignment_error': errors})
    return bool(errors)  # True = override was needed
```

3.3 Complaint AI Celery Task (complaints/tasks.py)

```
@shared_task(bind=True, max_retries=3)
def process_complaint_ai(self, complaint_id):
    complaint = Complaint.objects.get(id=complaint_id)
    for img in complaint.images.filter(image_type='complaint'):
        # LAYER 1: FAKE DETECTION
        for fn in [check_exif, check_ela, check_ai_generated]:
            result = fn(img.image.path)
            if not result['real']:
                complaint.status      = 'Rejected -- Fake Image'
                complaint.fake_detected = True
                complaint.fake_reason   = result['reason']
                complaint.save()
                send_push.delay(str(complaint.citizen_id),
                    'Complaint Rejected', 'Upload a real camera photo.')
                return
    # LAYER 2: YOLO
    confidence = YOLOValidator().validate(complaint.images.all())
    complaint.ai_confidence = confidence
    complaint.status = 'Validated' if confidence >= 0.5 else 'Pending Validation'
    complaint.save()
    DuplicateDetector().detect(complaint)  # same category+ward within 100m
    PriorityEngine().calculate(complaint)
    if complaint.status == 'Validated':
        route_complaint.delay(str(complaint_id))
```

3.4 Dept. Dashboard KPI View (dashboard/views.py)

```
class DeptKPIView(APIView):
    permission_classes = [IsDeptOfficer | IsAdmin]

    def get(self, request):
        # Admin: can pass ?dept_id=X, defaults to all
        # Officer: always filtered to their own department
        if request.user.role == 'admin':
            dept_id = request.query_params.get('dept_id')
        else:
            dept_id = request.user.department_id

        qs = Complaint.objects.all()
        if dept_id:
            qs = qs.filter(department_id=dept_id)

        # Optional filters
        ward_id    = request.query_params.get('ward_id')
        date_from  = request.query_params.get('date_from')
        date_to    = request.query_params.get('date_to')
        severity   = request.query_params.get('severity')
        if ward_id:   qs = qs.filter(ward_id=ward_id)
        if date_from: qs = qs.filter(submitted_at__gte=date_from)
        if date_to:   qs = qs.filter(submitted_at__lte=date_to)
        if severity:  qs = qs.filter(priority_level=severity)

        return Response({
            'total':       qs.count(),
            'pending':     qs.filter(status='Routed').count(),
            'in_progress': qs.filter(status='In-Progress').count(),
            'resolved':    qs.filter(status='Closed').count(),
            'rejected':    qs.filter(status__in=['Invalid','Rejected -- Fake Image']).count(),
            'avg_resolution_days': self._avg_resolution(qs),
        })
```

4. Python Packages (requirements.txt)

Package	Version	Purpose
Django	4.2.16	Web framework
djangorestframework	3.15.2	REST API
djangorestframework-simplejwt	5.3.1	JWT authentication
django-cors-headers	4.4.0	CORS for React-Django
django-environ	0.11.2	Load .env settings
psycopg2-binary	2.9.9	PostgreSQL driver
Pillow	10.4.0	Image EXIF reading
opencv-python-headless	4.10.0	ELA image analysis
numpy	1.26.4	Array math for ELA

transformers	4.44.0	HuggingFace fake detector
torch	2.3.0	ML runtime for transformers
ultralytics	8.2.0	YOLOv8 model
celery	5.4.0	Async task queue
redis	5.1.0	Redis client for Celery
firebase-admin	6.5.0	FCM push notifications
gunicorn	22.0.0	Production server

5. Deployment Checklist

Step 1

Create Render account at render.com (free). Create PostgreSQL 15 database. Copy DATABASE_URL.

Step 2

Create Redis Cloud account at redislabs.com. Create free 30MB database. Copy Redis URL.

Step 3

Create Firebase project. Enable Cloud Messaging. Download service account JSON. Get web app config for frontend.

Step 4

Push code to GitHub. Make sure .gitignore excludes: .env, __pycache__, node_modules, media/.

Step 5

Create Render Web Service (backend). Build: pip install -r requirements.txt && python manage.py migrate. Start: gunicorn resolveIT.wsgi --bind 0.0.0.0:8000.

Step 6

Create Render Background Worker (Celery). Same build. Start: celery -A resolveIT worker --loglevel=info.

Step 7

Create Render Static Site (frontend). Build: npm install && npm run build. Publish: dist/.

Step 8

Add all environment variables on Render: DATABASE_URL, REDIS_URL, SECRET_KEY, ALLOWED_HOSTS, FCM_KEY, FIREBASE_CREDENTIALS_PATH.

Step 9

Run seed command to create 4 categories, 4 wards, 4 departments, demo users: python manage.py seed_demo.

Step 10

Sign up at uptimerobot.com. Add HTTP monitor for Render backend URL. Interval: 10 minutes.

Step 11

Test complete flow: citizen submits -> AI validates -> dept officer assigns worker -> worker completes -> citizen verifies.